

**Автономная некоммерческая организация высшего образования
«Университет Иннополис»**

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(БАКАЛАВРСКАЯ РАБОТА)
по направлению подготовки
09.03.01 - «Информатика и вычислительная техника»**

**GRADUATION THESIS
(BACHELOR'S GRADUATION THESIS)
Field of Study
09.03.01 – «Computer Science»**

**Направленность (профиль) образовательной программы
«Информатика и вычислительная техника»
Area of Specialization / Academic Program Title:
«Computer Science»**

**Тема /
Topic**

**Исследование Kubernetes как платформы для бизнес-логики с
использованием Custom Resource Definitions и паттерна Operator /
Exploring Kubernetes as a Platform for Business Logic Using Custom
Resource Definitions and the Operator Pattern**

**Работу выполнил /
Thesis is executed by**

**Бобкунов Евгений /
Bobkunov Evgeny**

подпись / signature

**Руководитель
выпускной
квалификационной
работы /
Supervisor of
Graduation Thesis**

Dmitrii Creed

подпись / signature

**Консультанты /
Consultants**

подпись / signature

Иннополис, Innopolis, 2026

Contents

1	Introduction	4
1.1	Background and Context	4
1.2	Problem Statement	5
1.3	Research Questions	6
1.4	Research Objectives and Contributions	7
1.5	Research Methodology	8
1.6	Thesis Organization	9
1.7	Summary	10
2	Literature Review	11
2.1	Introduction	11
2.2	Kubernetes Architecture and Core Concepts	11
2.2.1	Container Orchestration and Kubernetes	11
2.2.2	Declarative API Model	12
2.2.3	Controller Pattern and Reconciliation Loop	13
2.3	Custom Resource Definitions	13
2.3.1	Extending the Kubernetes API	13
2.3.2	CRD Features and Capabilities	14
2.3.3	Performance and Scalability Considerations	14

2.4	The Operator Pattern	15
2.4.1	Operator Fundamentals	15
2.4.2	Operator Development Frameworks	15
2.4.3	Operator Use Cases and Applications	16
2.5	Related Systems and Approaches	16
2.5.1	Infrastructure as Code and GitOps	16
2.5.2	Service Mesh Architectures	16
2.6	Observability in Cloud-Native Systems	17
2.6.1	Monitoring and Metrics Collection	17
2.6.2	Distributed Tracing	17
2.7	Performance Studies and Challenges	18
2.7.1	Kubernetes Performance Characteristics	18
2.7.2	Complexity and Operational Considerations	18
2.8	Research Gaps	19
2.9	Summary	20

Bibliography cited	21
---------------------------	-----------

Chapter 1

Introduction

1.1 Background and Context

The rapid evolution of cloud-native computing has fundamentally transformed how modern software systems are architected, deployed, and operated. At the core of this transformation lies Kubernetes, which has emerged as the de facto standard for container orchestration since its open-source release in 2014 [1], [2]. Initially developed by Google based on their internal Borg system and later donated to the Cloud Native Computing Foundation (CNCF), Kubernetes provides robust capabilities for automating deployment, scaling, and management of containerized applications [3], [4].

However, Kubernetes offers far more than container orchestration capabilities. Its extensible architecture, built around a declarative API model and a control plane pattern, enables it to function as a universal control plane for managing arbitrary resources [5]. Through Custom Resource Definitions (CRDs) and the Operator pattern, Kubernetes can be extended to manage not only infrastructure components but potentially business logic itself [6]–[8]. This extensibility

has been thoroughly documented in both academic literature and industry best practices [9], [10].

Modern cloud systems strive for maximum automation and flexibility. Organizations are increasingly seeking unified platforms that can manage both infrastructure and application logic through consistent interfaces [11]. The declarative nature of Kubernetes, combined with its powerful extension mechanisms, positions it as a potential candidate for implementing business logic, not just orchestrating containers. This paradigm shift raises fundamental questions about the boundaries of the Infrastructure-as-Code approach and whether Kubernetes can serve as a unified control plane for both infrastructure and application concerns [12].

1.2 Problem Statement

Despite the widespread adoption of Kubernetes for infrastructure management, there remains a lack of systematic study regarding its applicability for implementing business logic using CRDs and the Operator pattern. Traditional application architectures typically separate business logic implementation from infrastructure concerns, with applications deployed as microservices that implement domain-specific functionality through conventional programming approaches.

Several projects have demonstrated the power of extending Kubernetes through CRDs and Operators, including Crossplane for infrastructure provisioning [13], Kubeflow for machine learning workflows, and various database operators [14]. However, most existing solutions focus on infrastructure management or specialized workloads rather than general-purpose business logic [15].

The main problem lies in understanding whether and how Kubernetes,

through CRDs and Operators, can effectively serve as a platform for implementing business logic. Key questions include: What are the performance characteristics of such an approach? How does it compare to traditional REST API-based architectures? What are the operational implications? What types of business problems are well-suited to this paradigm?

Solving this problem is important for several reasons. First, it could enable organizations to consolidate their technology stacks by using Kubernetes as a unified platform. Second, it may provide new patterns for building highly automated, self-healing business applications. Third, it could help determine clear boundaries for when this approach is beneficial versus when traditional architectures should be preferred.

1.3 Research Questions

This thesis investigates the following research questions:

RQ1: How can business logic be effectively modeled and implemented using Kubernetes Custom Resource Definitions and the Operator pattern?

RQ2: What are the performance characteristics of a business logic implementation based on CRDs and Operators compared to traditional REST API architectures?

RQ3: What are the operational implications and observability capabilities of using Kubernetes as a business logic platform?

RQ4: Under what circumstances is the CRD/Operator approach advantageous for implementing business logic, and what are its limitations?

1.4 Research Objectives and Contributions

The primary objective of this research is to explore and evaluate Kubernetes as a platform for implementing business logic through Custom Resource Definitions and the Operator pattern. Specific objectives include:

1. Study existing approaches to extending the Kubernetes API through CRDs and Operators, analyzing patterns and best practices established by the cloud-native community [9], [10].
2. Design and develop a prototype application that implements representative business logic using CRDs and Operators, demonstrating the feasibility of this approach.
3. Establish comprehensive observability for the prototype system using cloud-native tools including Prometheus and OpenTelemetry to enable detailed performance analysis [16]–[18].
4. Conduct experimental comparisons between the CRD/Operator-based implementation and a traditional architecture using PostgreSQL with REST API.
5. Analyze results across multiple dimensions including reaction time to events, implementation complexity, observability capabilities, and system reliability.

The expected contributions of this work include:

- A systematic analysis of using Kubernetes CRDs and Operators for business logic implementation, filling a gap in current research [11].

- A working prototype demonstrating practical patterns for modeling business entities and processes as Kubernetes custom resources.
- Quantitative performance comparisons providing empirical evidence of the strengths and weaknesses of this approach.
- Practical recommendations and guidelines for practitioners considering this architectural approach, including identification of suitable use cases and potential pitfalls.
- Insights into the boundaries of Kubernetes extensibility and the Infrastructure-as-Code paradigm.

1.5 Research Methodology

This research employs a mixed-methods approach combining system design, implementation, and experimental evaluation:

Phase 1 — Literature Review and Architecture Design: Conduct comprehensive review of existing literature on Kubernetes extensibility, CRDs, Operators, and related patterns [2], [8], [9]. Design the architecture for both the CRD/Operator-based system and the baseline traditional system.

Phase 2 — Implementation: Develop the prototype systems using appropriate tools and frameworks. The CRD/Operator implementation will utilize Kubebuilder for scaffolding and controller development [15]. Both systems will implement equivalent business logic to enable fair comparison.

Phase 3 — Observability Setup: Configure comprehensive monitoring using Prometheus for metrics collection and OpenTelemetry for distributed trac-

ing [16]–[18]. Implement GitOps practices using ArgoCD for deployment management [19], [20].

Phase 4 — Experimental Evaluation: Conduct controlled experiments measuring key performance indicators including reaction time to events, implementation complexity, observability coverage, and reliability [21].

Phase 5 — Analysis and Conclusions: Analyze experimental results, synthesize findings, and formulate conclusions on the applicability of the approach.

1.6 Thesis Organization

The remainder of this thesis is organized as follows:

Chapter 2 — Literature Review: Provides comprehensive background on Kubernetes architecture, Custom Resource Definitions, the Operator pattern, and related work in extending Kubernetes for business logic. Reviews relevant observability tools and GitOps practices.

Chapter 3 — System Design and Architecture: Describes the design of both the CRD/Operator-based system and the traditional baseline system. Details the business logic domain selected for implementation and architectural decisions.

Chapter 4 — Implementation: Covers implementation details including CRD specifications, Operator controller logic, observability instrumentation, and deployment configuration.

Chapter 5 — Experimental Evaluation: Presents the experimental setup, methodology, and results of performance comparisons between the two approaches.

Chapter 6 — Discussion: Analyzes results, discusses implications, identifies suitable use cases, and provides recommendations for practitioners.

Chapter 7 — Conclusion: Summarizes key findings, discusses limitations, and suggests directions for future work.

1.7 Summary

This chapter has introduced the research problem of using Kubernetes as a platform for business logic through Custom Resource Definitions and the Operator pattern. While Kubernetes has proven highly successful for infrastructure management, its applicability for implementing business logic remains understudied [11]. This thesis aims to address this gap through systematic design, implementation, and evaluation of both CRD/Operator-based and traditional architectures. The expected outcomes include practical insights into when and how this approach can be beneficial, contributing to the broader understanding of cloud-native application development patterns.

Chapter 2

Literature Review

2.1 Introduction

This chapter provides a comprehensive review of the literature and technologies relevant to exploring Kubernetes as a platform for business logic implementation. The chapter is organized into several sections covering Kubernetes architecture and extensibility, Custom Resource Definitions and Operators, related approaches to extending Kubernetes, observability in cloud-native systems, and performance considerations.

2.2 Kubernetes Architecture and Core Concepts

2.2.1 Container Orchestration and Kubernetes

Kubernetes, initially developed by Google and open-sourced in 2014, has become the leading platform for container orchestration [1], [2]. The system builds upon years of experience running production workloads at Google, combined

with best-of-breed ideas from the open-source community [3], [4]. Kubernetes automates deployment, scaling, and management of containerized applications, providing a platform-agnostic abstraction layer over underlying infrastructure.

The architecture of Kubernetes follows a control plane pattern, where a centralized control plane manages the desired state of the cluster, and worker nodes execute containerized workloads [2], [22]. The control plane consists of several key components: the API server, which exposes the Kubernetes API and serves as the entry point for all operations; etcd, a distributed key-value store that maintains cluster state; the scheduler, which assigns pods to nodes; and the controller manager, which runs controllers to reconcile actual state with desired state [10].

2.2.2 Declarative API Model

A distinguishing feature of Kubernetes is its declarative API model [5]. Rather than imperatively instructing the system on how to perform operations, users declare the desired state of resources, and Kubernetes automatically reconciles the actual state to match. This declarative approach enables powerful automation capabilities and forms the foundation for extending Kubernetes functionality [9].

The Kubernetes API is organized into API groups and versions, providing a structured approach to API evolution and extensibility [23]. Core resources like Pods, Services, and Deployments are part of the built-in API, while custom resources can be added through extension mechanisms. This design has been validated through extensive production use and is thoroughly documented in standard references [2], [4].

2.2.3 Controller Pattern and Reconciliation Loop

Kubernetes extensively uses the controller pattern for managing resources [10], [24]. A controller watches for changes to resources of interest and takes actions to reconcile the actual state with the desired state specified in the resource definition. This reconciliation loop runs continuously, enabling self-healing and automated management capabilities [8].

Controllers operate independently and asynchronously, communicating only through the Kubernetes API server. This architectural pattern enables high scalability and resilience, as controllers can be distributed across multiple instances and automatically recover from failures [9]. The controller pattern has become a fundamental building block for cloud-native applications and is extensively covered in Kubernetes design patterns literature [10].

2.3 Custom Resource Definitions

2.3.1 Extending the Kubernetes API

Kubernetes provides mechanisms for extending its API, with Custom Resource Definitions (CRDs) being the primary approach [9], [25]. CRDs represent a declarative approach to defining new resource types without requiring custom API server code, making them accessible to operators and developers [2].

A CRD defines a new resource type, including its API group, version, kind, and schema, making it a first-class citizen in the Kubernetes API [6], [9]. This approach enables extending Kubernetes to manage domain-specific resources while maintaining consistency with the platform's declarative model.

2.3.2 CRD Features and Capabilities

Modern CRDs support advanced features including schema validation using OpenAPI v3, multiple versions with conversion webhooks, status subresources for privilege separation, custom columns in kubectl output, and structural schemas for efficient storage and processing [9], [26]. These capabilities enable CRDs to provide rich, production-ready APIs comparable to built-in Kubernetes resources.

CRDs enable developers to model custom workflows, advanced configurations, and support for domain-specific requirements that go beyond Kubernetes' built-in resources. The flexibility of CRDs has made them a cornerstone of cloud-native extensibility, with hundreds of widely-used CRDs available in the ecosystem [11].

2.3.3 Performance and Scalability Considerations

The performance characteristics of CRDs depend heavily on the underlying etcd datastore, which stores all cluster state. Research on etcd performance shows that at scale, write latency can become a bottleneck, particularly for clusters with many CRD instances [27]. Organizations deploying CRD-intensive applications must carefully consider etcd capacity planning and implement appropriate monitoring.

2.4 The Operator Pattern

2.4.1 Operator Fundamentals

The Operator pattern represents a methodology for packaging, deploying, and managing Kubernetes applications by extending the Kubernetes API [7], [8]. Operators combine CRDs with custom controllers to encode operational knowledge about managing complex applications, effectively acting as automated site reliability engineers [14].

Operators capture the aim of human operators who manage services—they have deep knowledge of how systems should behave, how to deploy them, and how to react to problems. By codifying this operational knowledge, Operators enable applications to be self-managing, automatically handling tasks like upgrades, backups, scaling, and failure recovery [8], [10].

2.4.2 Operator Development Frameworks

Kubebuilder is the most prominent framework for facilitating Operator development [15]. Built on top of controller-runtime and controller-tools libraries, it provides scaffolding for creating CRDs and controllers using Go [9]. The framework automates much of the boilerplate code generation, allowing developers to focus on implementing business logic in the reconciliation loop.

Research on Operator development best practices emphasizes the importance of proper RBAC configuration, efficient handling of custom resources, and robust error handling [14]. Studies analyzing Operator bugs across Kubernetes operators have identified common failure patterns related to state management, error handling, and race conditions [28].

2.4.3 Operator Use Cases and Applications

Operators have been successfully applied to numerous domains including database management, monitoring systems, service meshes, and machine learning platforms [8], [14]. For infrastructure provisioning, Crossplane exemplifies using Operators to manage cloud resources declaratively through Kubernetes [13]. This approach enables treating infrastructure as Kubernetes resources, managed through the same workflows as application deployments.

2.5 Related Systems and Approaches

2.5.1 Infrastructure as Code and GitOps

Infrastructure as Code (IaC) provides declarative approaches to infrastructure provisioning [12]. While similar in declaring desired state, traditional IaC tools typically use imperative execution models and lack the continuous reconciliation capabilities inherent to Kubernetes controllers [5].

The GitOps methodology extends IaC principles by using Git as the single source of truth and implementing continuous delivery through pull-based reconciliation [19], [20]. GitOps shares philosophical alignment with Kubernetes' declarative model, making it a natural fit for managing Kubernetes resources including CRDs.

2.5.2 Service Mesh Architectures

Service mesh technologies like Istio demonstrate advanced usage of Kubernetes extensibility for managing service-to-service communication [29]. These

systems use CRDs extensively to define traffic management, security policies, and observability configurations, providing examples of complex operational logic implemented through Kubernetes extension mechanisms [30].

Research comparing service mesh architectures reveals trade-offs between feature richness and operational complexity, with implications for designing custom business logic controllers. The patterns used in service meshes offer lessons for separating concerns in business logic implementations.

2.6 Observability in Cloud-Native Systems

2.6.1 Monitoring and Metrics Collection

Observability in distributed systems encompasses metrics, logs, and traces. Prometheus has emerged as the de facto standard for metrics collection in Kubernetes environments [16], [18]. Its pull-based model, powerful query language (PromQL), and native Kubernetes service discovery make it well-suited for cloud-native monitoring [21].

For CRD-based applications, proper instrumentation of controllers and business logic is essential for understanding system behavior. Controllers should expose metrics about reconciliation loop performance, error rates, and business-specific indicators.

2.6.2 Distributed Tracing

OpenTelemetry represents the industry standard for distributed tracing and observability instrumentation [17]. Distributed tracing becomes particularly important when business logic spans multiple reconciliation loops and controllers,

as it enables tracking the cascading changes of objects through the Kubernetes control plane.

The integration of observability tools with Kubebuilder-based Operators is well-established, with frameworks providing automatic instrumentation of common controller metrics. However, domain-specific business metrics require custom implementation within the reconciliation logic.

2.7 Performance Studies and Challenges

2.7.1 Kubernetes Performance Characteristics

Studies have examined Kubernetes performance under various conditions. Research on etcd deployment impact demonstrates that control plane performance and application performance both depend critically on etcd performance characteristics [27]. Write latency and throughput limitations in etcd can become bottlenecks for CRD-intensive workloads.

2.7.2 Complexity and Operational Considerations

While Kubernetes provides powerful capabilities, it introduces significant complexity and operational overhead [1], [11]. Managing a production Kubernetes cluster requires expertise in networking, security, storage, and distributed systems. Organizations must weigh these operational costs against the benefits of Kubernetes' automation and standardization.

For business logic implementations specifically, additional complexity arises from the need to design appropriate CRD schemas, implement robust controllers,

and handle edge cases in the reconciliation logic [9]. The learning curve for Kubernetes development is substantial, potentially impacting development velocity.

2.8 Research Gaps

Despite extensive research on Kubernetes architecture, CRDs, and Operators, several gaps exist in current literature:

1. Limited systematic study of using Kubernetes for general-purpose business logic implementation, with most examples focused on infrastructure or specialized domains [11].
2. Insufficient quantitative performance comparisons between CRD/Operator-based approaches and traditional architectures for equivalent business logic.
3. Lack of comprehensive guidelines on when the CRD/Operator pattern is appropriate for business applications versus when traditional architectures should be preferred.
4. Limited research on implementation complexity and developer experience when using Kubernetes as a business logic platform [11].

This thesis aims to address these gaps through systematic design, implementation, and evaluation of Kubernetes-based business logic using CRDs and Operators.

2.9 Summary

This chapter has reviewed the relevant literature on Kubernetes architecture, extensibility mechanisms, and related technologies. Kubernetes provides a powerful platform for building cloud-native applications through its declarative API, CRDs, and Operator pattern [2], [4], [8]. While extensively used for infrastructure management, its application to general business logic remains understudied [11]. The next chapter describes the design of systems to explore this research question.

Bibliography cited

- [1] Kubernetes, *Production-grade container orchestration*, 2024. [Online]. Available: <https://kubernetes.io>.
- [2] M. Luksa, *Kubernetes in Action*, 1st. Manning Publications, 2017, ISBN: 9781617293726.
- [3] Cloud Native Computing Foundation, *Cloud native computing foundation*, 2024. [Online]. Available: <https://www.cncf.io>.
- [4] K. Hightower, B. Burns, and J. Beda, *Kubernetes: Up and Running: Dive into the Future of Infrastructure*, 1st. O'Reilly Media, 2017, ISBN: 978-1491935675.
- [5] J. L. Wofford, “Designing a scalable framework for declarative automation on distributed systems”, 2021. arXiv: [2104.13263](https://arxiv.org/abs/2104.13263) [cs.DC]. [Online]. Available: <https://arxiv.org/abs/2104.13263>.
- [6] Kubernetes, *Custom resources*, 2024. [Online]. Available: <https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/>.
- [7] ———, *Operator pattern*, 2025. [Online]. Available: <https://kubernetes.io/docs/concepts/extend-kubernetes/operator/>.

- [8] J. Dobies and J. Wood, *Kubernetes Operators: Automating the Container Orchestration Platform*. O'Reilly Media, 2020, ISBN: 978-1492048046.
- [9] M. Hausenblas and S. Schimanski, *Programming Kubernetes: Developing Cloud-Native Applications*. O'Reilly Media, 2019, ISBN: 978-1492047100.
- [10] B. Ibryam and R. Huß, *Kubernetes Patterns: Reusable Elements for Designing Cloud-Native Applications*, 1st. O'Reilly Media, 2020, ISBN: 978-1492050285.
- [11] S. I. Shamim, J. A. Gibson, P. Morrison, and A. Rahman, "Benefits, challenges, and research topics: A multi-vocal literature review of kubernetes", *arXiv preprint arXiv:2211.07032*, 2022. arXiv: [2211.07032 \[cs.SE\]](https://arxiv.org/abs/2211.07032). [Online]. Available: <https://arxiv.org/abs/2211.07032>.
- [12] M. Naumovska, *Infrastructure as code in devops: Benefits, tools, and best practices*, 2025. [Online]. Available: <https://www.microtica.com/blog/what-is-infrastructure-as-code>.
- [13] Crossplane, *Crossplane is the cloud-native framework for platform engineering*, 2024. [Online]. Available: <https://www.crossplane.io>.
- [14] S. Kanthed, "Openshift operators: Automating complex application lifecycle management", *International Journal of Latest Research Perspectives*, Feb. 2024. DOI: [10.70528/IJLRP.v5.i2.1436](https://www.ijlrp.com/research-paper.php?id=1436). [Online]. Available: <https://www.ijlrp.com/research-paper.php?id=1436>.
- [15] Kubernetes SIG, *Kubebuilder: Sdk for building kubernetes apis using crds*, 2024. [Online]. Available: <https://github.com/kubernetes-sigs/kubebuilder>.

- [16] Prometheus, *Prometheus - monitoring system & time series database*, 2024. [Online]. Available: <https://prometheus.io>.
- [17] OpenTelemetry, *Opentelemetry*, 2024. [Online]. Available: <https://opentelemetry.io>.
- [18] Y. Liu, Z. Yu, Q. Wang, H. Mei, G. Song, and H. Li, “Research on cloud-native monitoring system based on Prometheus”, in *Fourth International Conference on Sensors and Information Technology (ICSI 2024)*, vol. 13107, SPIE, 2024. DOI: [10.1117/12.3029320](https://doi.org/10.1117/12.3029320). [Online]. Available: <https://doi.org/10.1117/12.3029320>.
- [19] ArgoProject, *Argo cd - declarative gitops cd for kubernetes*, 2024. [Online]. Available: <https://argo-cd.readthedocs.io>.
- [20] GitLab, *What is gitops?*, 2023. [Online]. Available: <https://about.gitlab.com/topics/gitops/>.
- [21] N. Sukhija and E. Bautista, “Towards a framework for monitoring and analyzing high performance computing environments using kubernetes and prometheus”, Nov. 2019. DOI: [10.1109/SmartWorld-UIC-ATC-SCALCOM-IOP-SCI.2019.00087](https://doi.org/10.1109/SmartWorld-UIC-ATC-SCALCOM-IOP-SCI.2019.00087).
- [22] Kubernetes, *Cluster architecture*, 2024. [Online]. Available: <https://kubernetes.io/docs/concepts/architecture/>.
- [23] —, *The kubernetes api*, 2024. [Online]. Available: <https://kubernetes.io/docs/concepts/overview/kubernetes-api/>.
- [24] —, *Controllers*, 2024. [Online]. Available: <https://kubernetes.io/docs/concepts/architecture/controller/>.

- [25] ———, *Extending the kubernetes api*, 2024. [Online]. Available: <https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/>.
- [26] ———, *Extend the kubernetes api with custom resource definitions*, 2025. [Online]. Available: <https://kubernetes.io/docs/tasks/extend-kubernetes/custom-resources/custom-resource-definitions/>.
- [27] L. Larsson, W. Tärneberg, C. Klein, E. Elmroth, and M. Kihl, “Impact of etcd deployment on kubernetes, istio, and application performance”, 2020. arXiv: 2004.00372 [cs.NI]. [Online]. Available: <https://arxiv.org/abs/2004.00372>.
- [28] Q. Xu, Y. Gao, and J. Wei, “An empirical study on kubernetes operator bugs”, *ACM Digital Library*, 2024. DOI: [10.1145/3650212.3680396](https://doi.org/10.1145/3650212.3680396). [Online]. Available: <https://doi.org/10.1145/3650212.3680396>.
- [29] P. Josyula and A. Kumar, “Architectural trade-off analysis of modern service mesh designs: A qualitative atam study”, 2025. DOI: [10.1109/CICTN64563.2025.10932541](https://doi.org/10.1109/CICTN64563.2025.10932541).
- [30] R. Chandramouli and Z. Butcher, “Building secure microservices-based applications using service-mesh architecture”, *NIST Special Publication 800-204A*, Jan. 2020. DOI: [10.6028/NIST.SP.800-204A](https://doi.org/10.6028/NIST.SP.800-204A). [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-204A.pdf>.